ایمپورت کردن کتابخانه های اولیه

```
import sys
import re
import math
import pickle
import random
from collections import defaultdict, Counter
```

در این بخش ، کد ما کتابخانه ی اولیه و پایه ی کد رو ایمپورت میکنه مثل کتابخانه ی **"math"** که دستورات ریاضی جدید یا **'random'** برای ورودی های شانسی

تعریف کلاس ها

کلاس های اولیه

تعریف کلاس Config

```
class Config:

    DATASET_FILE = "dataset.txt"

    MODEL_FILE = "language_model.pkl"

    MAX_NGRAM = 4

    CACHE_SIZE = 5000
```

در این بخش از کد ما. برای راحت تر کردن کار ما، ما یک کلاس تعریف کردیم که یک سری متغیر مفید توش هست

تعریف کلاس Utilities

```
class Utils:
    @staticmethod

    def is_persian_char(ch):

        return '\u0600' <= ch <= '\u06FF'
    @staticmethod

    def remove_diacritics(text):
        arabic_diacritics = re.compile("""
                                    ّ    | # Tashdid
```

```
                                        َ    | # Fatha
                                        ً    | # Tanwin Fath
                                        ُ    | # Damma
                                        ٌ    | # Tanwin Damm
                                        ِ    | # Kasra
                                        ٍ    | # Tanwin Kasr
                                        ْ    | # Sukun
                                        ـ

                                    """, re.VERBOSE)

        return re.sub(arabic_diacritics, '', text)
    @staticmethod

    def normalize_numbers(text):

        persian_numbers = "۰۱۲۳۴۵۶۷۸۹"
        english_numbers = "0123456789"

        for p, e in zip(persian_numbers, english_numbers):

            text = text.replace(e, p)

        return text
```

توی این بخش ما برای راحت کردن کارمون اومدیم یک کلاس دیگه تعریف کردیم که خورده کاری ها و کار های غیر اصلی معیار سازی رو توش راحت انجام بدیم. اومدیم چند کار کردیم مثلا یک تابع برای تبدیل اعداد انگلیسی به فارسی یا مثلا حذف علائم صدادار مثل فتحه یا کسره

تعریف کلاس Normalizer

```
class PersianNormalizer:

    def __init__(self):
        self.char_map = {
            "ى" :"ي",
            "ک" :"ك",
            "ه" :"ۀ",
            "و" :"ؤ",
            "ا" :"إ",
            "ا" :"أ",
            "ه" :"ة"
        }

    def normalize_chars(self, text):

        for a, b in self.char_map.items():
            text = text.replace(a, b)
```

```
        return text

    def remove_extra_spaces(self, text):

        return re.sub(r"\s+", " ", text)

    def normalize_punctuation(self, text):

        text = text.replace("،", " ، ")
        text = text.replace("؟ " ,"؟ ")
        text = text.replace(".", " . ")

        return text

    def normalize(self, text):

        text = Utils.remove_diacritics(text)
        text = Utils.normalize_numbers(text)
        text = self.normalize_chars(text)
        text = self.normalize_punctuation(text)
        text = self.remove_extra_spaces(text)
        return text.strip()
```

برای اینکه کارمون رو راحت کنیم ، میایم یه کلاس تعریف میکنیم که بیشتر کار اصلاحات املایی ما رو انجام بده. مثل تبدیل حروف عربی مثل «ك» به نسخه ی فارسی «ک» و در اخر هم. با تعریف تابع **normalize()** بیشتر تابع هایی که قبلا در کلاس های دیگر و همین کلاس تعریف کردیم را در یک تابع ترکیب میکنیم

تعریف کلاس Tokenizer

```
class PersianTokenizer:

    def __init__(self):

        self.pattern = re.compile(r'\w+|[^\w\s]')

    def tokenize(self, text):

        tokens = self.pattern.findall(text)

        return tokens

    def detokenize(self, tokens):

        sentence = ""

        for tok in tokens:
            if tok in [".", "؟", "!", "،"]:
```

```
                sentence = sentence.rstrip() + tok + " "
            else:
                sentence += tok + " "

        return sentence.strip()
```

در این بخش برای راحت تر شدن کارمون میایم یه کلاس تعریف میکنیم، این کلاس دارای ابزار هایی است برای جدا کردن کلمات از هم. ما برای اصلاح املایی کلمات نیاز داریم که کلمات را جدا جدا و دانه دانه اصلاح املایی کنیم و این جدا سازی رو با این کلاس عملی میکنیم. در این کلاس دو تابع یکی برای جدا کردن کلمات **tokenize()** و یکی دیگر برای چسبوندن کلمات به هم **detokenize()** استفاده میکنیم

کلاس های پردازش اطلاعات

کش (Cache)

```
class LRUCache:

    def __init__(self, max_size=1000):

        self.max_size = max_size
        self.cache = {}
        self.order = []

    def get(self, key):

        if key not in self.cache:
            return None

        self.order.remove(key)
        self.order.append(key)

        return self.cache[key]

    def put(self, key, value):

        if key in self.cache:
            self.order.remove(key)
        elif len(self.cache) >= self.max_size:
            oldest = self.order.pop(0)
            del self.cache[oldest]

        self.cache[key] = value
        self.order.append(key)
```

در اینجا برای افزایش سرعت پردازش، ما یک کش تعریف . کش یک نوع حافظه ی فوق سریع هست که برای افزایش سرعت پردازش استفاده میشه ما اینجا از **LRUCache استفاده میکنیم یا **Least Recently Used Cache** این به

این معنا است که حافظه ی کش ما از پر کاربرد ترین عضو تا کم کاربرد ترین عضو مرتب میشود. برای اطلاعات بیشتر فایل Least Recently Used Cache را مطالعه کنید

لود کردن دیتاست

```
class DatasetLoader:

    def load_sentences(self, path):

        sentences = []

        with open(path, "r", encoding="utf-8") as f:

            for line in f:
                line = line.strip()

                if line:
                    sentences.append(line)

        return sentences
```

برای راحت کردن کارمون میایم از یک کلاس استفاده میکنیم، این کلاس خیلی چیز خاصی نیست فقط میاد و فایل dataset رو میخوانه و جمله جمله میکنه

استخراج داده از متن

```
class TextStatistics:

    def word_frequency(self, sentences):

        counter = Counter()

        for s in sentences:
            words = s.split()
            counter.update(words)

        return counter

    def vocabulary(self, sentences):

        vocab = set()

        for s in sentences:
            words = s.split()
            vocab.update(words)
```

```
        return vocab
```

برای راحت کردن کارمون میایم و یک کلاس درست میکنیم، این بخش از کد میاید و یک سری داده از فایل dataset که قبلا باز کردیم اسخراج میکنه و دو تابع جدید تعریف میکند

فاصله

```
class EditDistance:

    def distance(self, s1, s2):

        m = len(s1)
        n = len(s2)
        dp = [[0]*(n+1) for _ in range(m+1)]

        for i in range(m+1):
            dp[i][0] = i

        for j in range(n+1):
            dp[0][j] = j

        for i in range(1, m+1):
            for j in range(1, n+1):
                cost = 0 if s1[i-1] == s2[j-1] else 1
                dp[i][j] = min(
                    dp[i-1][j] + 1,
                    dp[i][j-1] + 1,
                    dp[i-1][j-1] + cost
                )

        return dp[m][n]
```

در این بخش از کد ما برای راحتی خودمان میایم یک کلاس تعریف میکنیم، این کلاس بسیار فهمش برام سخت بود ولی تونستم بفهمم و الان به صورت خلاصه توضیح میدم. به صورت ساده اگر بگم. کد ما میاید کم خرج ترین راه برای تبدیل یک کلمه به یک کلمه ی دیگر را محاسبه میکند. این کار رو با سه روش ، حذف، اضافه و جابجا کردن انجام میدهد. برای اطلاعات بیشتر Edit Distance را مطالعه کنید

کلاس مدل زبانی N-Gram

مدل زبانی N-Gram

```
class NGramLanguageModel:

    def __init__(self, n=4):

        self.n = n
```

```
        self.unigrams = Counter()
        self.bigrams = defaultdict(Counter)
        self.trigrams = defaultdict(Counter)
        self.fourgrams = defaultdict(Counter)
        self.total_words = 0
```

برای راحت کردن کارمون ما ما از یک کلاس استفاده میکنیم . همه ی بخش های بعدی جزئی از کلاس حساب میشوند ولی صرفا به دلیل بلند بودنشون جدا جدا بهشون پرداخته شده بگذریم. مدل زبانی **N-Gram** یعنی به وجود اوردن یک شبه هوش مصنوعی که بتواند تا **N** کلمه ی بعدی رو پیشبینی کند. برای اطلاعات بیشتر **N-Gram Language Model** را مطالعه فرمایید. در این بخش از کد صرفا چند متغیر تعریف میشود و کار خاصی انجام نمی شود

تمرین

```
    def train(self, sentences):

        tokenizer = PersianTokenizer()

        for sentence in sentences:
            tokens = tokenizer.tokenize(sentence)

            for i, word in enumerate(tokens):
                self.unigrams[word] += 1
                self.total_words += 1

                if i >= 1:
                    w1 = tokens[i-1]
                    self.bigrams[w1][word] += 1

                if i >= 2:
                    w1 = tokens[i-2]
                    w2 = tokens[i-1]
                    self.trigrams[(w1, w2)][word] += 1

                if i >= 3:
                    w1 = tokens[i-3]
                    w2 = tokens[i-2]
                    w3 = tokens[i-1]
                    self.fourgrams[(w1, w2, w3)][word] += 1
```

در این بخش از کلاس ما یک تابع **train** تعریف میکنیم که میاید با استفاده از تابع هایی که در بخش های پیشین تعریف کردیم تمام توکن ها رو به دسته های تکی، دوتایی،سه تایی و چهارتایی تقسیم میکند و انها را بر اساس فراوانی جمع بندی میکند

احتمال

```
    def unigram_prob(self, word):

        if word not in self.unigrams:
```

```
            return 1e-9

        return self.unigrams[word] / self.total_words

    def bigram_prob(self, w1, w2):

        if w1 not in self.bigrams:
            return self.unigram_prob(w2)

        count = self.bigrams[w1][w2]
        total = sum(self.bigrams[w1].values())

        if total == 0:
            return self.unigram_prob(w2)

        return count / total

    def trigram_prob(self, w1, w2, w3):

        key = (w1, w2)

        if key not in self.trigrams:
            return self.bigram_prob(w2, w3)

        count = self.trigrams[key][w3]
        total = sum(self.trigrams[key].values())

        if total == 0:
            return self.bigram_prob(w2, w3)

        return count / total

    def fourgram_prob(self, w1, w2, w3, w4):

        key = (w1, w2, w3)

        if key not in self.fourgrams:
            return self.trigram_prob(w2, w3, w4)

        count = self.fourgrams[key][w4]
        total = sum(self.fourgrams[key].values())

        if total == 0:
            return self.trigram_prob(w2, w3, w4)

        return count / total
```

**این بخش از کلاس احتمال وجود کلمات رو محاسبه میکند. این یعنی چی ؟ یعنی اینکه مثلا احتمال وجود «است» در trigram «این کتاب علی» را محاسبه کند. اگر هم نتوانست از تکنیک FallBack استفاده میکند. برا اطلاعات درباره ی

تکنیک fallback. فایل FallBack Strategy را مطالعه فرمایید

نمره دهی جملات

```
def sentence_probability(self, tokens):

    score = 0.0

    for i in range(len(tokens)):

        if i >= 3:

            prob = self.fourgram_prob(
                tokens[i-3],
                tokens[i-2],
                tokens[i-1],
                tokens[i]

            )
        elif i == 2:

            prob = self.trigram_prob(
                tokens[i-2],
                tokens[i-1],
                tokens[i]

            )

        elif i == 1:
            prob = self.bigram_prob(
                tokens[i-1],
                tokens[i]
            )

        else:
            prob = self.unigram_prob(tokens[i])
        score += math.log(prob + 1e-12)

    return score
```

در این بخش از کلاس ما از همون توابعی که در بخش احتمال تعریف کردیم استفاده میکنیم تا یک مجمع التوابعبه نام `sentence_probability` بسازیم که بتواند نمره بندی و محاسبه کند

پیشبینی کلمات بعدی

```
def predict_next(self, context):

    if len(context) >= 3:
```

```
            key = tuple(context[-3:])

            if key in self.fourgrams:
                return self.fourgrams[key].most_common(5)

        if len(context) >= 2:
            key = tuple(context[-2:])

            if key in self.trigrams:
                return self.trigrams[key].most_common(5)

        if len(context) >= 1:
            key = context[-1]

            if key in self.bigrams:
                return self.bigrams[key].most_common(5)

        return self.unigrams.most_common(5)
```

**این بخش از کد ما دقیقا مثل بخش احتمال هست اما بجای اینکه احتمال را محاسبه کند. خود کلمه رو پیدا میکنه. دوباره از همون سیستم fallback استفاده میکنه FallBack Strategy

ذخیره

```
def save(self, path):

        data = {
            "unigrams": self.unigrams,
            "bigrams": self.bigrams,
            "trigrams": self.trigrams,
            "fourgrams": self.fourgrams,
            "total_words": self.total_words
        }

        with open(path, "wb") as f:
            pickle.dump(data, f)
```

در اینجا خیلی کار خاصی انجام نمیده همه ی داده های کسب شده و یاد گرفته شده رو ذخیره میکند

اجرا

```
def load(self, path):

        with open(path, "rb") as f:

            data = pickle.load(f)

        self.unigrams = data["unigrams"]
```

```
        self.bigrams = data["bigrams"]
        self.trigrams = data["trigrams"]
        self.fourgrams = data["fourgrams"]
        self.total_words = data["total_words"]
```

در اینجای کد تابع لود کردن تعریف میشه که بسیار سادست و فقط تمام اطلاعات یاد گرفته شده رو دوباره لود میکنه

دستور چک کردن وجود کلمه

```
 def word_exists(self, word):
        return word in self.unigrams
```

یک دستور ساده برای چک کردن اینکه ایا کلمه ی مورد نظر در واژگان یاد گرفته شده وجود دارد یا ندارد

دستور واژگان

```
def vocabulary(self):
        return set(self.unigrams.keys())
```

یک دستور ساده برای خروجی گرفتن تمامی لغات یاد گرفته شده

کلاس تصحیح املایی هوشمند

تصحیح املایی هوشمند

```
class SmartSpellChecker:

    def __init__(self, language_model):

        self.lm = language_model
        self.edit_distance = EditDistance()
        self.cache = LRUCache(Config.CACHE_SIZE)
        self.common_mistakes = {
            "تو" :"طو",
            "می‌خواهم" :"میخوام",
            "می‌خواهم" :"میخام",
            "می‌خواهم" :"میخامم",
            "خواهش" :"خاهش",
            "خواهشاً" :"خاهشا",
            "مرسی" :"مرسیی",
            "خوبی" :"خوبیی",
            "چطوری" :"چطوریی",
            "سلام" :"سلامم",
            "کتاب" :"کتتاب",
            "مدرسه" :"مدرسهه",
            "دانشجو" :"دانشجوع",
            "دوست" :"دووست",
```

```
            "ممنونم" :"ممنونم"
        }
```

کلاس بعدی مهمی که باید به آن بپردازیم کلاس اصلاح املایی هستش که نسبتا آسون هستش. توی این بخش از کد یک سری متغیر تعریف میکنیم و یک سمپل (نمونه ی آزمایشی) از کلمات اشتباه و نسخه های درست آنها.

حذف تکرار حروف

```
    def remove_repeated_letters(self, word):

        return re.sub(r'(.)\1{2,}', r'\1', word)
```

این قسمت از کد فقط یک تابع تعریف میکند که هر کلمه ای که به آن داده شده را تلاش میکند بخشی ازش پیدا کند که دارای حرف های بیشتر از سه بار تعداد تکرار باشد و آنها رو تبدیل به یک بار میکند

تولید کاندید

```
    def generate_candidates(self, word, max_distance=2):

        vocab = self.lm.vocabulary()
        candidates = []

        for v in vocab:
            if abs(len(v) - len(word)) > max_distance:
                continue

            dist = self.edit_distance.distance(word, v)

            if dist <= max_distance:
                candidates.append(v)

        return candidates
```

ما در این بخش از کد تابعی تعریف میکنیم که یک کلمه را میگیرد و لیست کامل کلمات یاد گرفته شده میگیرد و نزدیک ترین کلمه به کلمه ی اصلی را با استفاده از طول کلمه و کلاس **Edit_Distance پیدا میکنیم

نمره دهی کاندید

```
    def score_candidate(self, context, candidate):

        tokens = context + [candidate]
        return self.lm.sentence_probability(tokens)
```

در این بخش از کد ما یک تابع تعریف میکنیم که براساس تولید کاندید و بقیه ی کار هایی که از قبل کردیم. بیاید و کلمه ی بعدی پیشنهادی رو با قبلش جمع کنه تا یک جمله به وجود بیاید

بهترین تصحیح

```
def best_correction(self, context, word):

    if word in self.common_mistakes:
        return self.common_mistakes[word]

    if self.lm.word_exists(word):
        return word

    word = self.remove_repeated_letters(word)
    cached = self.cache.get(word)

    if cached:
        return cached

    candidates = self.generate_candidates(word)

    if not candidates:
        return word

    best_word = word
    best_score = -1e9

    for cand in candidates:
        score = self.score_candidate(context, cand)

        if score > best_score:
            best_score = score
            best_word = cand

    self.cache.put(word, best_word)

    return best_word
```

این بخش از کد ما میاید و یک مجمع التوابع به نام `best_correction` تعریف میکنیم که از تمامی توابعی که از قبل تعریف شده استفاده میکند تا متن را تصحیح کند . مثلا `self.remove_repeated_letters(word)` برای حذف حروف تکراری. و در آخر بهترین کلمه را به عنوان یک کلمه در رشته ی `best_word` خروجی میدهد

تصحیح جمله

```
def correct_sentence(self, tokens):

    corrected = []
```

```
        for i, word in enumerate(tokens):
            context = corrected[-3:]
            new_word = self.best_correction(context, word)
            corrected.append(new_word)

        return corrected
```

در این بخش از کد ما دوباره یک تابع مجمع التوابع تعریف میکنیم که توش تیکه تیکه و توکن توکن در یک حلقه در جمله جلو میرود و آن را تصحیح میکند و در آخر یک لیست از توکن های تصحیح شده خروجی میدهد

کلاس پردازش متن فارسی

پردازش متن فارسی

```
class PersianTextProcessor:

    def __init__(self, language_model):

        self.normalizer = PersianNormalizer()
        self.tokenizer = PersianTokenizer()
        self.spell_checker = SmartSpellChecker(language_model)
```

این بخش از کد صرفا یک سری از توابعی که قبلا تعریف شده را کپی میکند در این کلاس فعال میکند. این کلاس که ما برای راحت کردن کارمون تعریف کردیم. برای خورده کاری ها و اماده سازی متن برای رسمی سازی است

استاندارد سازی متن

```
    def standardize_text(self, text):

        text = self.normalizer.normalize(text)
        text = self.fix_spacing(text)
        text = self.fix_half_spaces(text)

        return text
```

در این بخش از کد ما یک مجمع التوابع به نام `standardize_text` تعریف میکنیم که متن را ورودی بگیرد، معیار سازی اش کند و فاصله ها و نیم فاصله ها را درست کند و اخرش هم متن تصحیح شده را خروجی دهد

تصحیح فاصله‌گذاری

```
    def fix_spacing(self, text):

        text = re.sub(r'\s+([.,!?؟،])', r'\1', text)
        text = re.sub(r'([.,!?؟،])([^\s])', r'\1 \2', text)
        text = re.sub(r'\s+', ' ', text)
```

```
        return text
```

در این بخش از کد ما از یک تابع تعریف میکنیم. این تابع توی بخش قبل استفاده شده بود و کارش تصحیح فاصله هاست. کد اینجوری کار میکنه که متن رو ورودی میگیره بعد فاصله ها را درست میکنه و متن درسته شده رو خروجی میده

قوانین نیمفاصله

```
    def fix_half_spaces(self, text):

        text = re.sub(r'\bمی\s+', 'می', text)
        text = re.sub(r'\bنمی\s+', 'نمی', text)
        text = re.sub(r'\bبی\s+', 'بی', text)

        return text
```

در این بخش از کد. نیم فاصله ها اصلاح میشوند و نحوه ی کار کردن اینه که اول ورودی میگیره کلمه رو بعدش بین «می،نمی،بی» و بعدش رو یک نیم فاصله میگذاره وسطش

تقسیم جملات

```
    def split_sentences(self, text):

        sentences = re.split(r'[.!؟]', text)
        sentences = [s.strip() for s in sentences if s.strip()]

        return sentences
```

در این بخش از کد ما. کد یک کار بسیار ساده میکند. یک تابع تعریف میکند که بر اساس علائم پایان جمله متن را جمله جمله میکند!. یک متن میگیرد بعد بر اساس اون علائم متن را `split()` میکند و در اخر یک لیست از جملات خروجی میدهد

مجمع التوابع تصحیح جمله

```
    def correct_sentence(self, sentence):

        tokens = self.tokenizer.tokenize(sentence)
        corrected_tokens = self.spell_checker.correct_sentence(tokens)

        return self.tokenizer.detokenize(corrected_tokens)
```

در این بخش از کد یک مجمع التوابع تعریف میشه که کد رو تکه تکه ، تصحیح املایی و دوباره بهم میچسباند

مجمع التوابع خروجی متن نهایی

```
    def process_text(self, text):

        text = self.standardize_text(text)
        sentences = self.split_sentences(text)
        corrected_sentences = []

        for s in sentences:
            corrected = self.correct_sentence(s)
            corrected_sentences.append(corrected)

        final_text = " ".join(corrected_sentences)

        return final_text
```

این اخرین بخش کلاس هستش که تمام تابع های کلاس رو به هم میچسباند و متن نهایی را خروجی میدهد. در اصل: متن را جمله جمله میکند سپس تصحیح را انجام میدهد و دوباره جملات را بهم میچسباند و خروجی میدهد

کلاس یادگیری مدل

یادگیری مدل

```
class ModelTrainer:

    def __init__(self):

        self.normalizer = PersianNormalizer()
        self.tokenizer = PersianTokenizer()
```

در این کلاس ما ابزار های لازم برای تمرین دادن مدل زبانی رو فراهم میکنیم. مثل بقیه ی کلاس ها با آماده سازی کلاس شروع میکنیم

یادگیری مدل از دیتاست

```
    def train(self, dataset_path, model_path):

        loader = DatasetLoader()
        sentences = loader.load_sentences(dataset_path)
        normalized_sentences = []

        for s in sentences:
            s = self.normalizer.normalize(s)
            normalized_sentences.append(s)

        model = NGramLanguageModel(Config.MAX_NGRAM)
        model.train(normalized_sentences)
        model.save(model_path)
```

```
        return model
```

در این بخش از کد ما، ما یک مجمع التوابع تعریف میکنیم که با استفاده از توابعی که در کلاس مدل زبانی بوده، یک سیستم تمرین دهی را بر اساس فایل dataset انجام دهد

لود یا یادگیری

```
    def load_or_train(self):

        model = NGramLanguageModel(Config.MAX_NGRAM)

        try:
            model.load(Config.MODEL_FILE)

        except:
            model = self.train(
                Config.DATASET_FILE,
                Config.MODEL_FILE
            )

        return model
```

در این بخش از کد ما، کد سعی میکنه فایل های dataset رو لود کنه اگه نتونست فایل language_model.pkl و dataset رو باهم اجرا میکنه و تمرین میکنه و خروجی میده

کلاس رابط کاربری گرافیکی

آماده سازی رابط کاربری گرافیکی

```
from PySide6.QtWidgets import (
    QApplication,
    QMainWindow,
    QTextEdit,
    QPushButton,
    QFileDialog,
    QLabel,
    QVBoxLayout,
    QWidget,
    QMessageBox
)
from PySide6.QtGui import QFont

class PersianCorrectorGUI(QMainWindow):

    def __init__(self, processor):

        super().__init__()
```

```
        self.processor = processor
        self.setWindowTitle("Persian Smart Corrector")
        self.setGeometry(200, 200, 900, 600)
        self.init_ui()
```

این کلاس مخصوص نمایش دادن این همه کدی که زدیم هستش. ما از کتابخانه ی **pyside6** استفاده میکنیم و اولش آنها را تعریف میکنیم . سپس یک کلاس مینویسیم برای راحت کردن کارمان و چندین تابع را از قبل کپی میکنیم

ساخت رابط کاربری

```
    def init_ui(self):

        layout = QVBoxLayout()
        self.label_input = QLabel("متن ورودی:")
        self.text_input = QTextEdit()
        self.text_input.setFont(QFont("Tahoma", 12))
        self.label_output = QLabel("متن اصلاح شده:")
        self.text_output = QTextEdit()
        self.text_output.setFont(QFont("Tahoma", 12))
        self.text_output.setReadOnly(True)
        self.btn_correct = QPushButton("اصلاح متن")
        self.btn_save = QPushButton("ذخیره فایل")
        self.btn_correct.clicked.connect(self.correct_text)
        self.btn_save.clicked.connect(self.save_file)
        layout.addWidget(self.label_input)
        layout.addWidget(self.text_input)
        layout.addWidget(self.btn_correct)
        layout.addWidget(self.label_output)
        layout.addWidget(self.text_output)
        layout.addWidget(self.btn_save)
        container = QWidget()
        container.setLayout(layout)
        self.setCentralWidget(container)
```

طوری که میبینید یک سلسله مراتب داخل این تابع هست که رابط کاربری رو میسازی که به ترتیب این کار رو میکنه: مقدار دهی تابع، نوشتن یک متن ، مقدار دهی تابع ، تنظیم فونت، نوشتن یک متن ، مقدار دهی یک تابع ، تنظیم فونت ، نمایش این صفحه ، رسم یک دکمه با یک متن ، رسم یک دکمه ی دیگر ، ذخیره ی ورودی ، اضافه کردن همه ی این چیز هایی که گفتم.

گرفتن ورودی و پردازش

```
    def correct_text(self):

        text = self.text_input.toPlainText().strip()

        if not text:
            QMessageBox.warning(self, "متنی وارد نشده است" ,"خطا")

            return
```

```
        try:

            corrected = self.processor.process_text(text)
            self.text_output.setPlainText(corrected)

        except Exception as e:

            QMessageBox.critical(self, "خطا", str(e))
```

در این بخش از کد ما، اون ورودی ای که توی پنجره گرفتیم را پردازش و معیار سازی میکنیم. یک تابع تعریف میکنیم که اول متن را ساده سازی میکند (تکه تکه) سپس اگر متنی اصلا وارد نشده باشه هیچی خروجی نمیده و فقط ارور میده. سپس تلاش میکنه که متن را رسمی و معیار سازی کند ، اگر نتونست، یه پیام با متن (خطا) بدهد

ذخیره فایل

```
    def save_file(self):

        text = self.text_output.toPlainText()

        if not text:
            QMessageBox.warning(self, "متنی برای ذخیره وجود ندارد" ,"خطا")

            return

        file_path, _ = QFileDialog.getSaveFileName(
            self,

            "ذخیره فایل",
            "",
            "Text Files (*.txt)"
        )

        if not file_path:
            return

        try:

            with open(file_path, "w", encoding="utf-8") as f:

                f.write(text)

            QMessageBox.information(self, "فایل ذخیره شد" ,"موفق")

        except Exception as e:

            QMessageBox.critical(self, "خطا", str(e))
```

در این بخش از کد ما متن را ذخیره و هندل میکند، چندین حالت های اشتباه را هندل و حتی دیرکتوری را وردی میگیرد

نقطه ی درگاه اصلی

```
def main():
    # 1) آموزش یا بارگذاری مدل زبانی

    print("در حال آماده‌سازی مدل زبانی...")
    trainer = ModelTrainer()
    language_model = trainer.load_or_train()
    print("مدل زبانی آماده شد.")

    # 2) ساخت پردازشگر متن

    processor = PersianTextProcessor(language_model)

    # 3) راه‌اندازی GUI

    app = QApplication(sys.argv)
    window = PersianCorrectorGUI(processor)
    window.show()

    sys.exit(app.exec())
```

در این بخش از کد ما یک سوپر فوق مجمع التوابع تعریف میکنیم که تمام کاری که توی این تقریبا 1000 خطی که نوشتیم را در یک تابع خلاصه کند.

خود کد (بلاخره بهش رسیدیم >:O)

```
if __name__ == "__main__":
    main()
```

این اولین بخشی از کد هستش که درحال تعریف تابع نیستیم. و آخرین بخش کد هستش که فقط . میگه اگر کد ران شد تابع main رو ران کن. همین. بعد از این فایل سخن من با اساتید